

Compiladores: Panorama General y Contexto

Fases del Proceso de Compilación

DRA. MELISSA BRIGITTHE HINOJOSA CABELLO

Universidad Nacional Autónoma de México

Facultad de Ingeniería

03 de Septiembre de 2026





Hasta ahora hemos visto al compilador como una caja negra y descrito:

- ¿Qué es?
- ¿Por qué existe?
- ¿Sobre qué tipo de máquinas se ejecutan los programas?
- ¿Qué tipo de procesamiento existe?

Pero... **un compilador no es una caja negra**, es un sistema estructurado y está compuesto por una serie de análisis y transformaciones bien definidas, no estrictamente secuenciales que pueden combinarse u omitirse para cumplir un propósito específico:

Modificar la representación de un programa, mas no su comportamiento,
hasta obtener una forma que pueda ejecutarse en una máquina de ejecución.



A partir de ahora nos enfocaremos en la serie de análisis y transformaciones que se realizan sobre un programa para su posterior ejecución, respondiendo a las preguntas:

- ▶ ¿Qué ocurre dentro de un compilador?
- ▶ ¿Cómo se transforma un programa paso a paso?



Tarea 2: Fases del Proceso de Compilación

→ Investigar acerca de las fases del proceso de compilación:

- Análisis léxico
- Análisis sintáctico
- Análisis semántico
- Representación intermedia
- Optimización

→ Elaborar un reporte con un resumen de los puntos anteriores (máximo 2 páginas).

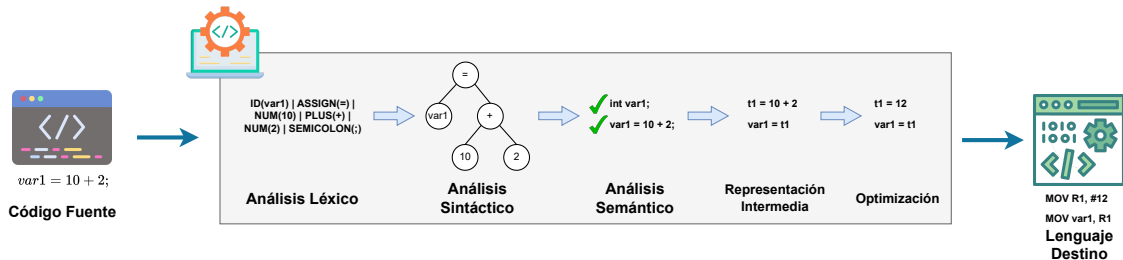
→ Entrega: **Martes 1ro. de Septiembre** (antes de la clase).

- Enviar reporte (digital) por correo.
- Discusión en clase.

Fases del proceso de compilación



El proceso de compilación se organiza en varias fases: primero, el código fuente es analizado y luego es transformado progresivamente hasta producir código ejecutable. Estas fases no siempre están completamente separadas; algunos compiladores pueden combinarlas para transformar poco a poco el programa hacia una representación más cercana a su ejecución.





El **análisis léxico** es la primera interacción real del compilador con el programa escrito en lenguaje de alto nivel: recibe una secuencia de instrucciones, tal como están escritas en el código fuente y **su objetivo es darle forma** a dichas instrucciones.

¿En qué consiste el *darle forma* a las instrucciones del código fuente?

Consiste en:

- ▶ Agrupar los caracteres en ***tokens***.
- ▶ Identificar qué partes del programa corresponden a
 - Palabras clave
 - Identificadores
 - Números
 - Operadores
 - Delimitadores
- ▶ Eliminar información irrelevante para las fases posteriores
 - Espacios en blanco
 - Saltos de línea
 - Comentarios



El resultado de esta fase es **una secuencia de tokens e información básica** asociada a cada token (tipo, valor, posición en el código) que será utilizada por el análisis sintáctico para verificar la estructura del programa.

Ejemplo: `var1 = 10 + 2;`
¿Qué elementos contiene esta instrucción?



A partir de este punto, utilizaremos una notación estándar para representar los tokens. Cada token se representará por una tupla $\langle \textit{tipo}, \textit{atributo} \rangle$ formada por un **tipo**, que indica la categoría léxica, y un **atributo**, que contiene información adicional cuando es necesaria. En el caso de los operadores o palabras reservadas no es necesario añadir información adicional.

Entonces, ¿cómo se vería la representación de los tokens del ejemplo anterior?

Ejemplo: `var1 = 10 + 2;`



Recordatorio:

- ▶ El análisis léxico transforma una secuencia de caracteres en una secuencia de tokens.
- ▶ El análisis léxico no entiende la estructura del programa, solo reconoce símbolos válidos del lenguaje.

Entonces, en esta fase todavía no se verifica:

- Si el programa está bien estructurado.
- Si las expresiones tienen sentido.
- Si las variables existen.



El análisis léxico recibe como entrada una secuencia de caracteres y debe producir una secuencia de tokens como salida. Un error léxico ocurre cuando no es posible convertir parte de esa entrada en ningún token válido. Ejemplos:

- Uso de caracteres ilegales

```
x = 10 @ 3;
```

- Números mal formados

```
y = 12.3.4;
```

- Identificadores inválidos

```
1var = 5;
```

- Cadenas sin cerrar

```
msg = "Hola mundo;
```

- Comentarios sin cerrar

```
/* inicio de comentario
```

```
x = 10;
```

- Uso incorrecto de notación numérica

```
z = 09;
```

- Uso de operadores inexistentes

```
a = b ** c;
```



El **análisis sintáctico** verifica que la secuencia de tokens producida por el análisis léxico cumpla con la gramática del lenguaje; es decir, que pueda organizarse conforme a las reglas del lenguaje y **construir una representación estructurada (válida)** del programa.

¿En qué consiste la construcción de una representación estructurada del programa?

Consiste en:

- ▶ Identificar cómo se agrupan los tokens.
- ▶ Pasar de una secuencia lineal de tokens a una estructura jerárquica.
- ▶ Representar explícitamente la dependencia entre elementos.
- ▶ Generar una forma intermedia que pueda ser utilizada por las fases posteriores del compilador.

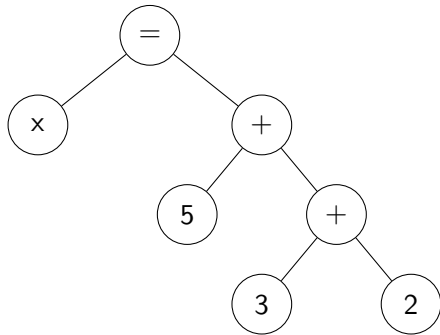
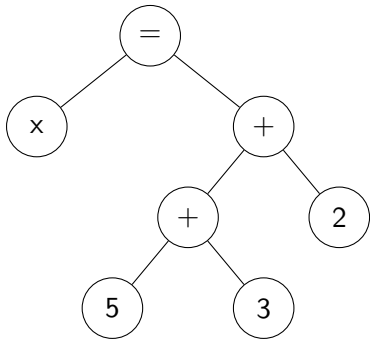


Si la secuencia de tokens es correcta, mediante el análisis sintáctico se construye una estructura que representa cómo está organizado el programa y define explícitamente qué se evalúa primero y cómo se agrupan los tokens.

Ejemplo: `var1 = 10 + 2;`
¿Cómo se vería la representación estructurada de esta instrucción?

$x = 5 + 3 + 2;$

→ ¿Cuál es la representación estructurada correcta?





En Autómatas vieron que una gramática genera cadenas; en Compiladores, esa gramática define la estructura de los programas.

¿Qué es una gramática?

Gramática

Es un conjunto de reglas que describe cómo se pueden construir cadenas válidas en un lenguaje, determinando:

- Cómo se agrupan los operadores.
- Cómo se forman las expresiones.
- Cómo se construye el árbol sintáctico.



Asociatividad a la izquierda

Cuando hay varios operadores del mismo nivel, i.e., con la misma precedencia, éstos se agruparán de izquierda a derecha.

$\langle \text{expresión} \rangle \rightarrow \langle \text{expresión} \rangle + \text{número} \mid \text{número}$

¿Qué elementos componen $\langle \text{expresión} \rangle$?

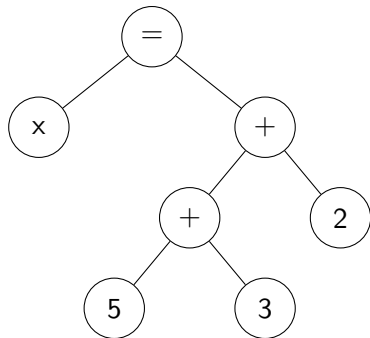
¿Qué indica esta regla?

$\langle \text{expresión} \rangle \rightarrow \text{número} + \text{número} \mid \text{número}$

Esta regla indica que una expresión puede ser otra expresión más un término (o número), o simplemente un término.

Entonces, ¿cómo se evalúa un ejemplo con asociatividad a la izquierda? ¿Cómo se ve el árbol sintáctico? Ejemplo: $x = 5 + 3 + 2$

$x = (5 + 3) + 2$



Dado este contexto, ¿es

$\langle \text{expresión} \rangle \rightarrow \langle \text{expresión} \rangle + \text{número} \mid \text{número}$
equivalente a

$\langle \text{expresión} \rangle \rightarrow \text{número} + \langle \text{expresión} \rangle \mid \text{número}?$



¿Es válido el árbol que genera $\langle \text{expresión} \rangle \rightarrow \text{número} + \langle \text{expresión} \rangle \mid \text{número}$?

→ Matemáticamente es equivalente porque la suma es asociativa en aritmética.

→ Pero sintácticamente no sería equivalente, ni correcto, si la gramática define la suma como asociativa a la izquierda.

En Compiladores, no es una decisión matemática, es una decisión sintáctica dependiente de la gramática. Aunque la suma sea matemáticamente asociativa, los árboles no son equivalentes entre sí.

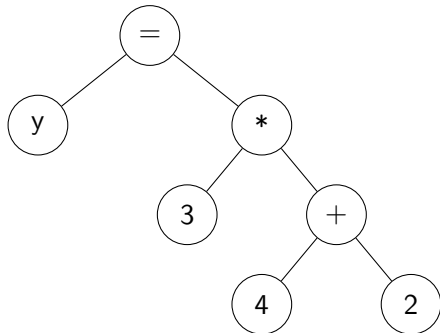
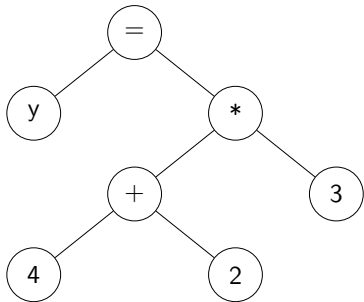
Entonces, la **asociatividad** define cómo se agrupan los operadores con el mismo nivel de precedencia. Para operaciones como suma, resta, multiplicación y división, los lenguajes de programación son asociativos a la izquierda.

En el ejemplo $x = 5 + 3 + 2$, el operador $\langle + \rangle$ es binario, de precedencia fija y asociativo a la izquierda. Por ello, el árbol sintáctico se construye de forma incremental, evaluando $5 + 3$, tomando su resultado y sumándole 2.

Asociatividad	Categoría	Operadores
Izquierda	Aritméticos	+ - * / %
	Relacionales	< <= > >=
	Igualdad	== !=
	Lógicos	&&
Derecha	Aritméticos	^
	Asignación	= += -= *= /=
	Ternario	?:

$y = (4 + 2) * 3;$

→ ¿Cuál es la representación estructurada correcta?



→ Los paréntesis obligan a que primero se construya el sub-árbol de la suma.



¿El árbol sintáctico tiene que ser estrictamente binario?

No necesariamente, pero en Compiladores un árbol binario ayuda a modelar explícitamente la asociatividad porque los operadores están definidos como binarios en la gramática, i.e., representa toda la gramática.

Ventajas:

- ▶ Es más simple de procesar en análisis semántico.
- ▶ Facilita la generación de código intermedio.
- ▶ Simplifica la evaluación recursiva de la gramática.

→ El árbol es una representación fiel del proceso de derivación gramatical.



El análisis sintáctico recibe como entrada una secuencia de tokens y verifica que dicha secuencia pueda organizarse conforme a la gramática del lenguaje. Un error sintáctico ocurre cuando los tokens son válidos individualmente, pero no pueden combinarse siguiendo las reglas del lenguaje. Ejemplos:

- Falta de símbolos obligatorios

`x = 10 + 3`

- Paréntesis desbalanceados

`y = (12 + 3;`

- Orden incorrecto de tokens

`x = 5 * + 2;`

- Estructuras de control mal formadas

`if x > 3) { ... }`



Hasta ahora, el compilador ya verificó que los símbolos existan (análisis léxico) y que estén bien organizados según la gramática (análisis sintáctico). Después, el **análisis semántico** se encarga de verificar que el programa tenga sentido desde el punto de vista del lenguaje.

¿Qué significa que el programa *tenga sentido*?

- ▶ Que las variables se usen después de ser declaradas.
- ▶ Que las operaciones se apliquen a tipos compatibles.
- ▶ Que las expresiones produzcan valores válidos.



El resultado de esta fase es la verificación del significado del programa, utilizando la estructura sintáctica y la información del contexto (tipos, declaraciones, alcance), para asegurar que las operaciones sean coherentes y válidas.

Ejemplo:

```
int var1;
```

```
var1 = 10 + 2;
```

¿Qué aspectos semánticos se verifican en estas instrucciones?



Recordatorio:

Análisis Léxico → ¿qué símbolos existen?

vs.

Análisis Sintáctico → ¿cómo se agrupan esos símbolos?

vs.

Análisis Semántico → ¿el programa tiene sentido?

El análisis semántico no cambia la forma del programa, pero sí puede rechazarlo si se detectan inconsistencias de significado.



Tarea 3: Fases del Proceso de Compilación

→ Ejercicios:

- `int a; float b;`
`a = b * (3 + 2) - 4;`
- `int x;`
`x = (y + 2) * (3 - 1);`
- `int a; int b;`
`a = (b + 3) > 5 + 2;`
- `bool flag; int x;`
`x = flag + 4 * 2;`
- `int a; float b; float c;`
`c = (a + 2) * (b - 3) / (a > 1);`
- `int x; float y; bool flag;`
`if (x + flag > 3) {`
 `z = y + 2;`
`}`
- `int i; float total;`
`for (i = 0; i < 10.5; i = i + 1) {`
 `total = total + i;`
`}`
`flag = total;`



→ Resolver los ejercicios.

Análisis Léxico

- Escribir secuencia de tokens.
- Determinar si existe error léxico.

Análisis Sintáctico

- Escribir reglas sintácticas.
- Determinar la representación estructurada.
- Determinar si existe error sintáctico.

→ Entrega: - -/**Septiembre/2026** (antes de la clase).

- Enviar ejercicios (capturas de pantalla, fotografías legibles o escaneado) por correo.
- Discusión en clase.

Análisis Semántico

- Verificar significado del programa.
- Determinar si existe error semántico.



Hasta ahora el compilador verificaba estructura y significado, y se respondían preguntas como:

- ▶ ¿Los símbolos son válidos? → **Análisis léxico**
- ▶ ¿La estructura es correcta? → **Análisis sintáctico**
- ▶ ¿El programa tiene sentido? → **Análisis semántico**

¿Por qué era así? ¿Cuál era el objetivo?

→ **Objetivo:** Detectar errores.



Desde el análisis léxico y hasta el análisis semántico, el compilador solamente actúa como un verificador que evalúa:

- Forma
- Coherencia
- Consistencia

Sin embargo, en estas 3 fases aún no se modifica el programa ni se genera una versión alternativa funcional. A partir de la fase de **representación intermedia**, el compilador deja de analizar y detectar errores para comenzar a generar el programa que se ha de ejecutar en una máquina de ejecución. El objetivo es construir una representación ejecutable derivada de:

- Transformaciones
- Reorganización
- Simplificaciones
- Preparación para la ejecución.



¿Qué es el código intermedio o representación intermedia?

Código Intermedio (IR)

Es una representación abstracta del programa que modela su comportamiento de manera independiente de la arquitectura de destino, permitiendo realizar transformaciones y optimizaciones antes de la generación de código final.

¿Por qué es una representación abstracta?

Porque no es el programa original. Pero tampoco es código máquina.

→ Es una estructura interna del compilador que elimina detalles superficiales del lenguaje fuente.



Esta representación intermedia se genera después del análisis semántico y corresponde a una representación estructurada y explícita del significado original del programa. Además, sirve como base para la optimización y generación del código final, simplificando construcciones complejas, introduciendo variables temporales y manteniendo **equivalencia semántica** con el programa en código fuente.

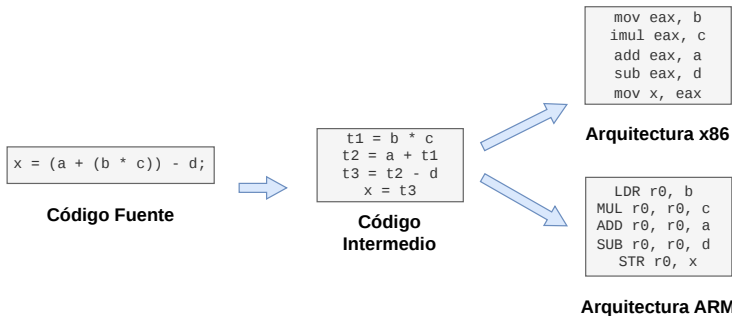
→ Si el programa fuente produce cierto resultado, el código intermedio debe producir exactamente el mismo resultado.

Representación intermedia



El código intermedio desacopla el significado del programa de la máquina que lo ejecutará, separando la semántica del programa de las restricciones físicas de la arquitectura de destino. De esta forma es independiente de:

- ▶ Registros físicos.
- ▶ Sistema operativo.





Durante la fase de **optimización**, el compilador busca mejorar la eficiencia del programa, pero sin modificar su comportamiento: aunque el código sea más rápido o use menos recursos, el resultado final debe ser exactamente el mismo que si no se aplicara ninguna optimización.

→ La optimización no cambia lo que el programa hace, sino cómo lo hace internamente.

Existen diversas técnicas de optimización, entre las cuales destacan:

- ▶ Eliminación de código muerto.
- ▶ Propagación de constantes.
- ▶ Simplificación de expresiones.
- ▶ Eliminación de subexpresiones comunes.

Ejemplos:

- Eliminación de código muerto.

```
int x = 10;  
int y = 5;  
int z = x + y;  
return x;
```

- Propagación de constantes.

```
int a = 5;  
int b = a + 3;
```



Ejemplos:

- Simplificación de expresiones.

```
int x = 3 * 4;
```

- Eliminación de subexpresiones comunes.

```
int x = (a + b) * 2;
```

```
int y = (a + b) + 5;
```



Cabe resaltar que, dependiendo del diseño del compilador, la optimización de código no necesariamente ocurre en una sola etapa. Ésta puede aplicarse sobre el código intermedio, durante la generación de código en lenguaje destino o incluso después, ajustando a detalles específicos de la arquitectura, eliminando instrucciones redundantes o mejorando el uso de registros.

Dado que la optimización de código se considera opcional, un compilador puede funcionar perfectamente sin realizar optimizaciones; no obstante, el código generado será menos eficiente por lo que en la práctica es altamente recomendable aplicar optimización.

Generación de código



Durante el proceso de compilación existen tres representaciones principales de código:

- ▶ Lenguaje fuente → *Programa escrito por el desarrollador en un lenguaje de alto nivel.*
- ▶ Código intermedio → *Representación interna del compilador, independiente de la máquina de ejecución.*
- ▶ Lenguaje destino → *Representación equivalente de instrucciones en ensamblador, código máquina o bytecode.*

```
#include <stdio.h>

int main() {
    int a = 5;
    int b = 3;
    int c = a + b;
    printf("Resultado: %d\n", c);
    return 0;
}
```



```
t1 = 5
t2 = 3
t3 = t1 + t2
c = t3
param c
call printf
return 0
```



```
mov DWORD PTR [rbp-4], 5
mov DWORD PTR [rbp-8], 3

mov eax, DWORD PTR [rbp-4]
add eax, DWORD PTR [rbp-8]

mov DWORD PTR [rbp-12], eax

mov esi, eax
lea rdi, format
xor eax, eax
call printf

mov eax, 0
ret
```



En la fase de **generación de código** el compilador toma el código intermedio y lo traduce para un lenguaje y sistema destino específicos. Es decir, el código resultante sí dependerá de la arquitectura, por lo que es necesario conocer y adaptar dicho código con base en los registros disponibles, instrucciones soportadas y modelo de memoria para esa arquitectura en particular.

Hasta antes de esta fase se ha trabajado con representaciones internas del compilador; en la generación de código, el programa escrito en lenguaje de alto nivel finalmente adopta una forma que puede ejecutarse en una arquitectura de hardware o en una máquina virtual.



Compilar no es solo traducir instrucciones; implica analizar, transformar, optimizar y adaptar un programa escrito en lenguaje fuente a una arquitectura física o máquina de ejecución específica.

► Código fuente

- Escrito por el programador.
- Portable.
- Independiente del hardware.

► Código intermedio

- Representación interna del compilador.
- Independiente de la arquitectura.
- Permite optimización avanzada.

► Código en lenguaje destino

- Específico de una arquitectura.
- Usa registros reales.
- Es ensamblador o bytecode.

► Ejecutable

- Código máquina binario.
- Totalmente dependiente del hardware.
- Listo para ejecutarse por el CPU o la máquina virtual.



Tarea 4: Fundamentos formales de análisis léxico y símbolos

→ Investigar acerca de:

- Lenguajes regulares
- Expresiones regulares
- Autómatas finitos deterministas
- Autómatas finitos no deterministas

→ Elaborar un reporte con las definiciones de los puntos anteriores.



→ Responder brevemente (con sus palabras) las siguientes preguntas:

- ¿Cuál es la relación entre expresiones regulares y autómatas finitos deterministas?
- ¿Por qué el análisis léxico puede modelarse con un AFD?
- ¿Qué ocurriría si un token requiriera memoria no acotada para reconocerse?

→ Entrega: - -/**Septiembre/2026** (antes de la clase).

- Enviar reporte (digital) por correo.
- Discusión en clase.